

Disaster Alert AI System – Initial Setup Guide

This document contains the complete initial setup plan for the Disaster Alert AI System project. The project is designed as a scalable multi-disaster early warning platform using Machine Learning, Deep Learning, Weather APIs, and Geospatial Intelligence. The first disaster target will be Flood Prediction because it provides the best balance of: - data availability - prediction accuracy - scalability - real-world usefulness - ML learning value The system is designed to run on a normal CPU-based ASUS laptop without requiring a dedicated GPU.

Project Vision

The final system will support multiple disasters:

- Floods
- Cyclones
- Wildfires
- Landslides
- Tsunami alerts
- Air pollution forecasting
- Asteroid risk monitoring

Architecture philosophy:

Separate specialized ML models for each disaster connected to one central intelligence platform.

Phase 1 Goal

Build an advanced Flood Prediction and Risk Alert System using historical weather and flood datasets.

Core outputs:

- Flood probability prediction
- Flood severity estimation
- Area risk scoring
- Future expansion into real-time alerts

Recommended Tech Stack

Programming Language:

- Python 3.11

Core Libraries:

- pandas
- numpy
- scikit-learn
- xgboost
- lightgbm
- matplotlib

- plotly
- jupyter
- fastapi

Deep Learning (later phases):

- tensorflow-cpu

Database:

- SQLite initially
- PostgreSQL later

Deployment (future):

- Docker
- Render / Railway / AWS

Why CPU-Friendly Setup?

Since the laptop does not have a dedicated GPU, the project setup is optimized for CPU training.

Important:

- Avoid training huge transformer models initially.
- Use XGBoost and Random Forest first.
- TensorFlow CPU version is enough for learning and prototyping.
- Most flood prediction datasets are tabular and work extremely well on CPU systems.

Recommended IDE Setup

Recommended IDE:

- VS Code

Recommended Extensions:

- Python
- Jupyter
- Pylance
- GitHub Copilot (optional)

Folder Structure:

project/

■

■■■ data/

■■■ notebooks/

■■■ models/

■■■ api/

■■■ frontend/

■■■ outputs/

■■■ requirements.txt

■■■ README.md

Python Installation

Install:

- Python 3.11
 - During installation check:
- [x] Add Python to PATH

Verify installation using:

```
python --version  
pip --version
```

Virtual Environment Setup

Commands:

Windows:

```
python -m venv venv
```

Activate:

```
venv\Scripts\activate
```

Upgrade pip:

```
python -m pip install --upgrade pip
```

Library Installation

Main installation command:

```
pip install -r requirements.txt
```

The provided requirements.txt file includes all important initial libraries.

Initial ML Pipeline

Step 1:

Dataset collection

Step 2:

Data cleaning

Step 3:

Feature engineering

Step 4:

Model training

Step 5:

Evaluation

Step 6:
Prediction engine

Step 7:
Alert generation

Best Initial ML Models

Use these models first:

1. Random Forest
2. XGBoost
3. LightGBM

Why?

- Extremely strong for tabular data
- CPU-friendly
- Fast training
- High accuracy
- Used in real-world forecasting systems

Future Deep Learning Expansion

Later phases may include:

- LSTM networks
- CNN satellite analysis
- Transformer forecasting models
- Real-time sensor fusion

These are advanced stages after the initial ML system works correctly.

Datasets Strategy

Initial datasets:

- Kaggle Flood Prediction datasets
- Rainfall datasets
- Weather history datasets

Advanced datasets later:

- NASA
- NOAA
- IMD (India Meteorological Department)
- Satellite weather feeds

Real-Time System Future Plan

Future architecture:

Weather APIs



Prediction Engine



Risk Analysis



Alert Generator



Dashboard / Mobile App

Performance Tips for Normal Laptop

- Close unnecessary apps during training
- Use smaller batch sizes
- Start with smaller datasets
- Save trained models regularly
- Use Jupyter notebooks for experimentation
- Prefer XGBoost over heavy neural networks initially
- Upgrade RAM before GPU if possible

Project Roadmap

PHASE 1:

Flood prediction ML model

PHASE 2:

Real-time weather integration

PHASE 3:

Risk scoring engine

PHASE 4:

Dashboard and visualization

PHASE 5:

Multi-disaster support

PHASE 6:

Mobile alerts and deployment

Initial Setup Complete ■

The project is now ready for dataset collection and model training.